

Laboratório de Concorrência · Aula 10

Gabriel Campos Feiten · RA 24047639-2 · Turma A

Gerado em 09/10/2026, 20:41:04 · tempo registrado no cronômetro: 16:20

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

A sincronização de cooperação acontece quando duas ou mais pessoas trabalham juntas para alcançar um objetivo em comum, organizando suas ações para que todos consigam realizar suas tarefas. Um exemplo é quando colegas organizam uma festa e cada um fica responsável por uma parte, como a comida, a decoração e a música. Já a sincronização de competição ocorre quando duas ou mais pessoas disputam um objetivo ou recurso, e suas ações precisam ser coordenadas para evitar conflitos. Um exemplo é quando duas pessoas querem usar o mesmo campo para jogar futebol e precisam combinar horários para que ambas possam utilizá-lo.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

2.000.000

1ª execução (n = 1.000.000)

1.714.260

2ª execução (n = 1.000.000)

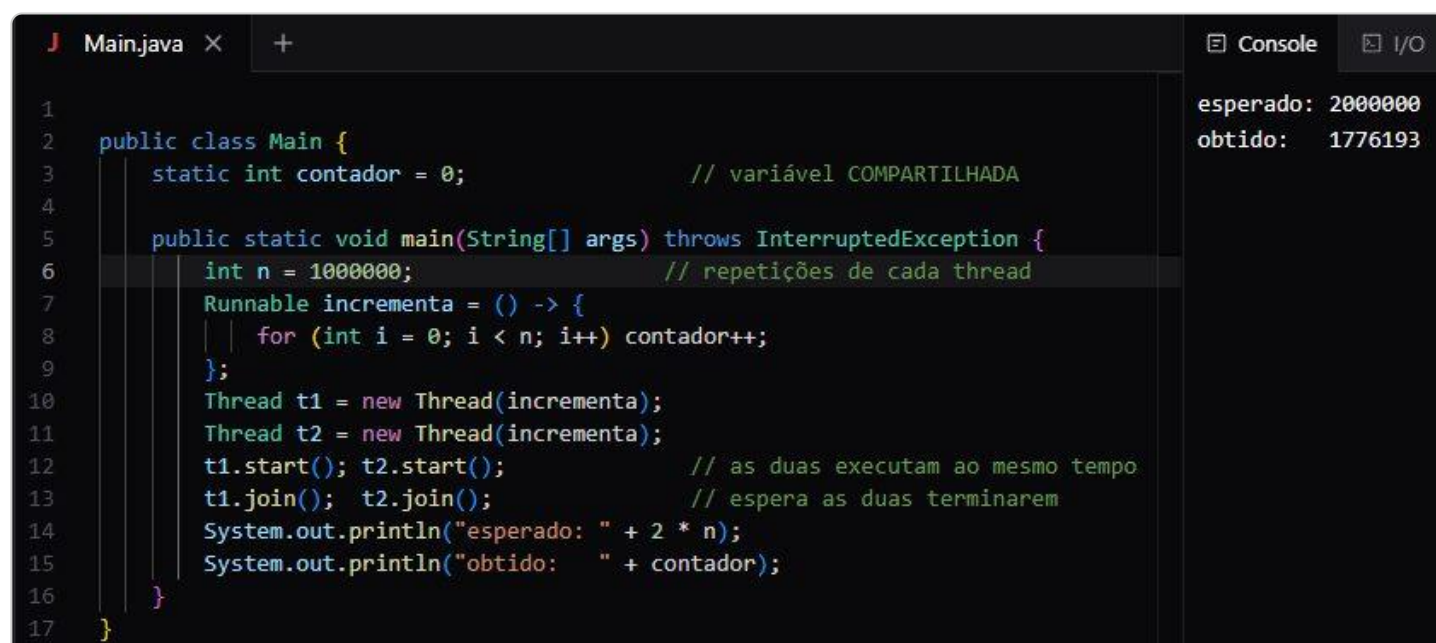
1.314.411

3ª execução (n = 1.000.000)

1.095.683

Saída do programa

não respondido



```
1 public class Main {
2     static int contador = 0; // variável COMPARTILHADA
3
4     public static void main(String[] args) throws InterruptedException {
5         int n = 1000000; // repetições de cada thread
6         Runnable incrementa = () -> {
7             for (int i = 0; i < n; i++) contador++;
8         };
9         Thread t1 = new Thread(incrementa);
10        Thread t2 = new Thread(incrementa);
11        t1.start(); t2.start(); // as duas executam ao mesmo tempo
12        t1.join(); t2.join(); // espera as duas terminarem
13        System.out.println("esperado: " + 2 * n);
14        System.out.println("obtido: " + contador);
15    }
16 }
17 }
```

Console I/O

esperado: 2000000
obtido: 1776193

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Os valores referentes a 1.000.000 não foram o esperado em nenhuma das 3 execuções, visto que, as threads utilizam a mesma variável e faz com que ela entre em condição de corrida, o que causa a alteração.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Ambas podem ler o mesmo valor, exemplo, lê 5, somam 1 e gravam 6, o contador que era para ser atualizado de 5 para sete somente vai de 5 para 6 e um incremento é perdido.

1.3 Com $n = 1000$, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Com o valor referente a 1000, nas duas execuções resultaram no mesmo valor esperado. Essa variação de resultados torna esse erro perigoso em sistemas reais porque ele pode não aparecer durante os testes, mas ocorrer em produção quando várias threads acessam os mesmos dados simultaneamente. Assim, o programa pode apresentar resultados incorretos de forma imprevisível, causando, por exemplo, inconsistências em saldos bancários, estoques ou registros de usuários.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

ABABAB → 6

AABBAB → 6

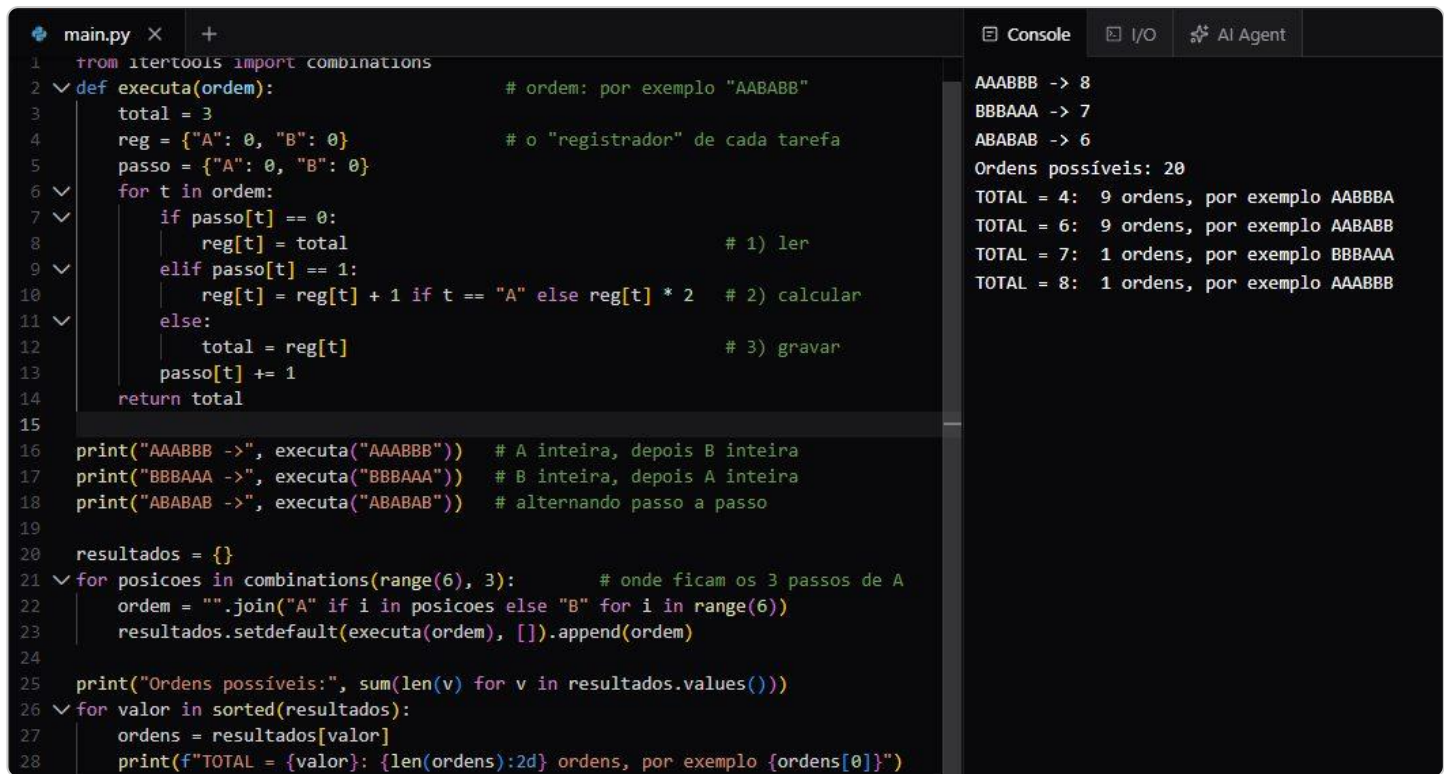
BBBAAA → 7

BABBAA → 4

ABBBA → 4

Saída do programa

não respondido



```

1 from itertools import combinations
2 def executa(ordem):
3     total = 3
4     reg = {"A": 0, "B": 0}
5     passo = {"A": 0, "B": 0}
6     for t in ordem:
7         if passo[t] == 0:
8             reg[t] = total
9         elif passo[t] == 1:
10            reg[t] = reg[t] + 1 if t == "A" else reg[t] * 2
11        else:
12            total = reg[t]
13            passo[t] += 1
14    return total
15
16 print("AAABBB ->", executa("AAABBB"))
17 print("BBBAAA ->", executa("BBBAAA"))
18 print("ABABAB ->", executa("ABABAB"))
19
20 resultados = {}
21 for posicoes in combinations(range(6), 3):
22     ordem = "".join("A" if i in posicoes else "B" for i in range(6))
23     resultados.setdefault(executa(ordem), []).append(ordem)
24
25 print("Ordens possíveis:", sum(len(v) for v in resultados.values()))
26 for valor in sorted(resultados):
27     ordens = resultados[valor]
28     print(f"TOTAL = {valor}: {len(ordens):2d} ordens, por exemplo {ordens[0]}")

```

AAABBB -> 8
BBBAAA -> 7
ABABAB -> 6
Ordens possíveis: 20
TOTAL = 4: 9 ordens, por exemplo AABBBB
TOTAL = 6: 9 ordens, por exemplo AABABB
TOTAL = 7: 1 ordens, por exemplo BBBAAA
TOTAL = 8: 1 ordens, por exemplo AAABBB

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

São 20 ordens possíveis e 4 resultados diferentes. É somente 1 ordem, a AAABBB.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

AABBBB, ocorre porque A lê e calcula, então B lê o valor, calcula e salva o resultado, por fim A grava o resultado da sua operação sobre o valor que B escreveu.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

Acredito que a monitor seja a mais rápida, já que os dados e operações ficam juntas. Acredito que todos irão funcionar.

Saída do programa

não respondido

The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a 'Main' class with a static method 'mede' that runs two threads in parallel. One thread increments a 'semCuidado' counter, while the other increments an 'AtomicInteger' counter. The 'main' method calls 'mede' three times to compare the execution times of these two approaches. The console output on the right shows the results: 'esperado' is 2000000, 'sem sincronizacao' took 1505805 ms, 'synchronized' took 2000000 ms, and 'AtomicInteger' took 2000000 ms.

```

1 import java.util.concurrent.atomic.AtomicInteger;
2
3 public class Main {
4     static int semCuidado = 0, comSync = 0;
5     static AtomicInteger atomico = new AtomicInteger();
6
7     static synchronized void incSync() { comSync++; } // uma thread por vez
8
9     static long mede(Runnable corpo) throws InterruptedException {
10         Thread a = new Thread(corpo), b = new Thread(corpo);
11         long inicio = System.nanoTime();
12         a.start(); b.start(); a.join(); b.join();
13         return (System.nanoTime() - inicio) / 1_000_000; // milissegundos
14     }
15
16     public static void main(String[] args) throws InterruptedException {
17         int n = 1_000_000;
18         long ms1 = mede(() -> { for (int i = 0; i < n; i++) semCuidado++; });
19         long ms2 = mede(() -> { for (int i = 0; i < n; i++) incSync(); });
20         long ms3 = mede(() -> { for (int i = 0; i < n; i++) atomico.incrementAndGet(); });
21         System.out.println("esperado: " + 2 * n);
22         System.out.println("sem sincronizacao: " + semCuidado + " em " + ms1 + " ms");
23         System.out.println("synchronized: " + comSync + " em " + ms2 + " ms");
24         System.out.println("AtomicInteger: " + atomico.get() + " em " + ms3 + " ms");
25     }
26 }

```

Console Output:

```

esperado: 2000000
sem sincronizacao: 1505805 em 45 ms
synchronized: 2000000 em 107 ms
AtomicInteger: 2000000 em 22 ms

```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

Synchronized e AtomicInteger deram o valor correto, porque sem verificação normalmente é mais rápida do que um algoritmo que precisa verificar e sincronizar.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não. Usar um AtomicInteger em cada saldo garante que cada operação individual seja indivisível, mas não garante que a transferência inteira aconteça corretamente. O dinheiro pode ser retirado de uma conta e, antes de ser depositado na outra, ocorrer uma falha, deixando os saldos inconsistentes. Para resolver isso, é necessário sincronizar a operação completa de transferência, utilizando, por exemplo o synchronized, garantindo que a retirada e o depósito aconteçam juntos, sem interferência de outras operações.

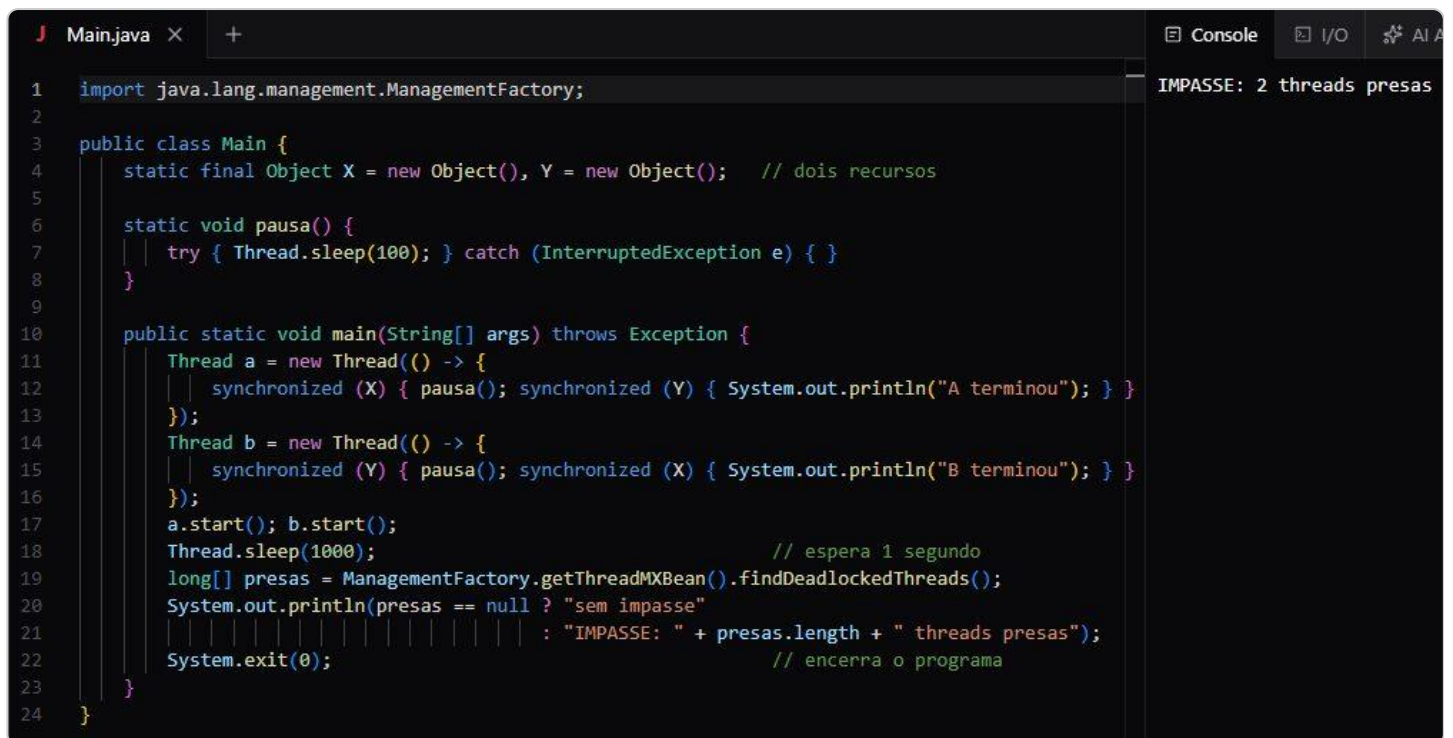
Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

Não, acho que vai dar deadlock

Saídas (antes e depois da correção)

não respondido

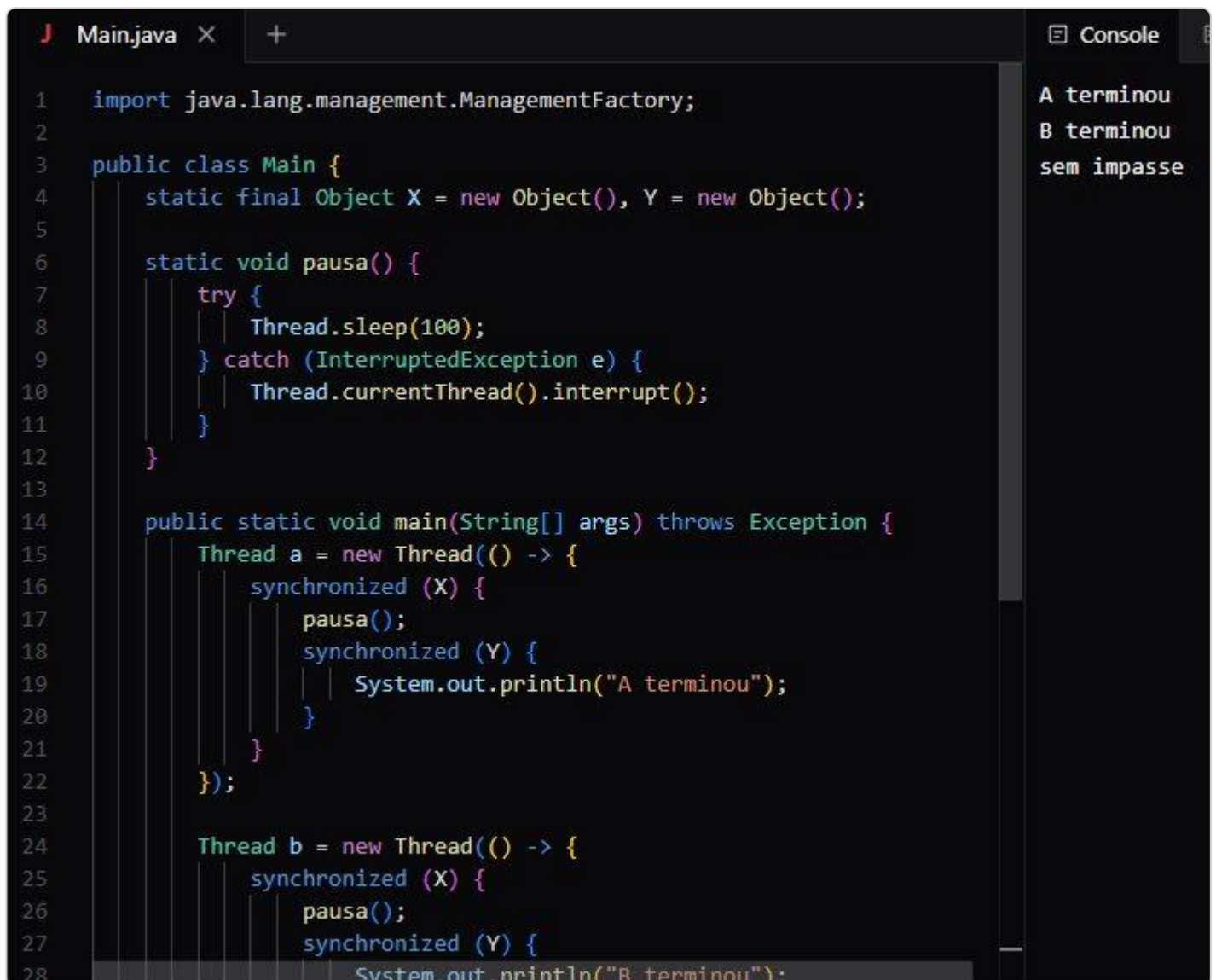


```

1  import java.lang.management.ManagementFactory;
2
3  public class Main {
4      static final Object X = new Object(), Y = new Object();    // dois recursos
5
6      static void pausa() {
7          try { Thread.sleep(100); } catch (InterruptedException e) { }
8      }
9
10     public static void main(String[] args) throws Exception {
11         Thread a = new Thread(() -> {
12             synchronized (X) { pausa(); synchronized (Y) { System.out.println("A terminou"); } }
13         });
14         Thread b = new Thread(() -> {
15             synchronized (Y) { pausa(); synchronized (X) { System.out.println("B terminou"); } }
16         });
17         a.start(); b.start();
18         Thread.sleep(1000);                                     // espera 1 segundo
19         long[] presas = ManagementFactory.getThreadMXBean().findDeadlockedThreads();
20         System.out.println(presas == null ? "sem impasse"
21             : "IMPASSE: " + presas.length + " threads presas");
22         System.exit(0);                                         // encerra o programa
23     }
24 }

```

IMPASSE: 2 threads presas



```

1  import java.lang.management.ManagementFactory;
2
3  public class Main {
4      static final Object X = new Object(), Y = new Object();
5
6      static void pausa() {
7          try {
8              Thread.sleep(100);
9          } catch (InterruptedException e) {
10             Thread.currentThread().interrupt();
11          }
12      }
13
14     public static void main(String[] args) throws Exception {
15         Thread a = new Thread(() -> {
16             synchronized (X) {
17                 pausa();
18                 synchronized (Y) {
19                     System.out.println("A terminou");
20                 }
21             }
22         });
23
24         Thread b = new Thread(() -> {
25             synchronized (X) {
26                 pausa();
27                 synchronized (Y) {
28                     System.out.println("B terminou");

```

A terminou
B terminou
sem impasse

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

As duas threads ficaram presas porque cada uma adquiriu um recurso e ficou esperando o recurso que estava com a outra. A thread A adquiriu X e esperava por Y, enquanto a thread B adquiriu Y e esperava por X.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Pegar os recursos sempre na mesma ordem evita que uma thread espere por um recurso preso por outra, eliminando esse tipo de impasse. Um exemplo é um sistema bancário em que duas transferências acessam as mesmas contas; bloquear as contas sempre na mesma ordem evita que as operações fiquem presas esperando uma pela outra.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

espero com 3 seja 200 ms, com 1 espero 600 ms e com 6 espero 100 ms.

Tempo com 1 trabalhador

600

Tempo com 3 trabalhadores

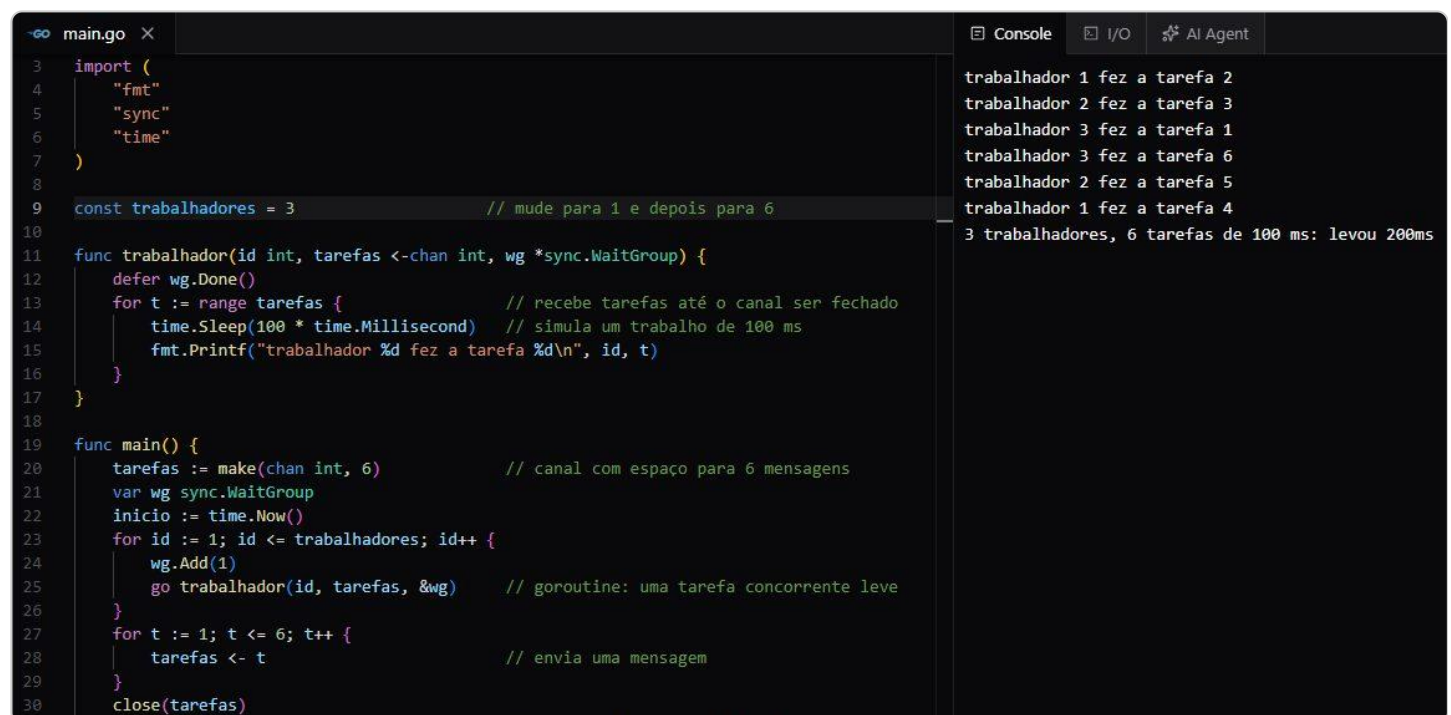
200

Tempo com 6 trabalhadores

100

Saída do programa

não respondido



```
main.go x
3 import (
4     "fmt"
5     "sync"
6     "time"
7 )
8
9 const trabalhadores = 3 // mude para 1 e depois para 6
10
11 func trabalhador(id int, tarefas <-chan int, wg *sync.WaitGroup) {
12     defer wg.Done()
13     for t := range tarefas { // recebe tarefas até o canal ser fechado
14         time.Sleep(100 * time.Millisecond) // simula um trabalho de 100 ms
15         fmt.Printf("trabalhador %d fez a tarefa %d\n", id, t)
16     }
17 }
18
19 func main() {
20     tarefas := make(chan int, 6) // canal com espaço para 6 mensagens
21     var wg sync.WaitGroup
22     inicio := time.Now()
23     for id := 1; id <= trabalhadores; id++ {
24         wg.Add(1)
25         go trabalhador(id, tarefas, &wg) // goroutine: uma tarefa concorrente leve
26     }
27     for t := 1; t <= 6; t++ {
28         tarefas <- t // envia uma mensagem
29     }
30     close(tarefas)
31
32     wg.Wait()
33     fim := time.Now()
34     fmt.Printf("3 trabalhadores, 6 tarefas de 100 ms: levou %v\n", fim.Sub(inicio))
35 }
```

Console

```
trabalhador 1 fez a tarefa 2
trabalhador 2 fez a tarefa 3
trabalhador 3 fez a tarefa 1
trabalhador 3 fez a tarefa 6
trabalhador 2 fez a tarefa 5
trabalhador 1 fez a tarefa 4
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Os três tempos dependem da quantidade de trabalhadores. Com 1 trabalhador, as 6 tarefas são executadas em sequência, levando aproximadamente 600 ms. Com 3 trabalhadores, as tarefas são distribuídas entre eles, levando cerca de 200 ms. Com 6 trabalhadores, cada um pode executar uma tarefa ao mesmo tempo, levando cerca de 100 ms.

ms. Na prática, os tempos podem ser um pouco maiores devido ao processamento e ao escalonamento. O programa não precisa de `synchronized` nem de travas porque utiliza canais (channels) para distribuir as tarefas entre os trabalhadores, evitando que várias goroutines precisem modificar diretamente a mesma variável compartilhada. O `WaitGroup` garante que o programa espere todos terminarem antes de encerrar.

Saídas (antes e depois da troca)

não respondido

```

1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int)           // canal SEM espaço (sem buffer)
9     ch <- 1                       // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }

```

fatal error: all goroutines are asleep - deadlock!

goroutine 1 [chan send]:
main.main()
/box/main.go:9 +0x38
exit status 2

Debug this error

```

1 // Prática 5b - Um canal sem ninguém do outro lado
2 // Cole em https://onecompiler.com/go
3 package main
4
5 import "fmt"
6
7 func main() {
8     ch := make(chan int, 1)       // canal SEM espaço (sem buffer)
9     ch <- 1                       // envia... e espera alguém receber
10    fmt.Println(<-ch)
11 }

```

1

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O primeiro programa trava porque o canal não tem espaço para armazenar mensagens. Ao executar `ch <- 1`, o programa fica esperando que alguém receba o valor, mas o recebimento só aconteceria na linha seguinte, que nunca é executada. Com `make(chan int, 1)`, o canal consegue armazenar uma mensagem. Assim, o valor 1 é enviado sem esperar por um receptor imediato e, em seguida, é recebido e impresso normalmente.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

Não será imprimido nenhum valor

Saídas (o erro e o resultado com move)

não respondido

Saída do programa

não respondido

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

Com move, a thread recebe uma cópia do valor de contador e altera apenas essa cópia. A variável original de main continua valendo 0, por isso o programa imprime 0.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Em Java foi descoberto após a execução, já em Rust foi antes da mesma. A vantagem do Rust é oferecer mais segurança antes da execução; o custo é que o programador precisa seguir regras mais rigorosas de propriedade e compartilhamento de dados.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A, D, B, C

Saída do programa

não respondido

The screenshot shows a code editor with a file named 'index.js'. The code is as follows:

```

1 // Prática 7 - JavaScript: uma única thread e um laço de eventos
2 // Cole em https://onecompiler.com/nodejs
3 console.log("A");
4 setTimeout(() => console.log("B"), 0);
5 Promise.resolve().then(() => console.log("C"));
6 console.log("D");
7
8 function ocupado(ms) {                // laço que NÃO devolve o controle
9   const fim = Date.now() + ms;
10  while (Date.now() < fim) { }
11 }
12
13 setTimeout(() => {
14   const t0 = Date.now();
15   setTimeout(() => console.log("E: timer de 10 ms rodou depois de " + (Date.now() - t0)), 10);
16   ocupado(300);                      // trava a única thread por 300 ms
17 }, 0);

```

The right side of the editor shows a 'Preview' tab with a message: 'No server started. The program exited without listening on a port. Check the console below.' Below this, the console output is shown:

```

A
D
C
B
E: timer de 10 ms rodou depois de 300 ms

```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A, D, C, B. A letra B, mesmo com tempo de 0 ms, saiu depois de D e C porque `setTimeout(..., 0)` não executa imediatamente, ele agenda uma tarefa para depois. Já D é executado diretamente e C é uma microtask de Promise, que tem prioridade sobre as tarefas de timer.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O timer de 10 ms executou depois desse tempo porque a função `ocupado(300)` bloqueia a única thread por aproximadamente 300 ms, impedindo que o Node.js execute outras tarefas durante esse período. Isso significa que, em um servidor Node.js, um cálculo pesado pode bloquear o processamento de outras requisições, atrasando as respostas de vários usuários ao mesmo tempo.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com `synchronized`

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode ocorrer uma condição de corrida (race condition), em que as duas pessoas conseguem comprar o último ingresso ao mesmo tempo, causando uma venda duplicada. Para evitar isso, é necessário sincronizar as operações

no banco de dados, utilizando uma transação com bloqueio ou uma atualização atômica que só permita a compra se ainda houver ingresso disponível.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi perceber que dois processos executados ao mesmo tempo podem gerar erros inesperados, mesmo quando o código parece estar correto.